Two-Way Fixed Effects

Two-way fixed effects on [panel data is a handy method for establishing linear models from time series data. To keep our notations consistent, we will use the term multivariate time series to refer to panel data in the following content.

Two-way Fixed Effects Model

A two-way fixed effects model is a linear model that allows the parameters to vary across both time and the variables[1],

$$y_{it} = \beta X_{it} + \alpha_i + \gamma_t + \epsilon_{it},$$

where α_i and γ_t represent the effect coming from the variable and time, respectively.

Example

To help readers outside of econometrics or causal inference get started with this model, we will use a simple example to illustrate the idea. We will construct a naive dataset with three groups and two variables linearly related to each other.

Data and Model Results

We construct a naive dataset that contains three articles (column `name`), each having a different distribution of prices and demand, while all of them are generated with the same linear relation between the variable `log_demand` and `log_price`. The data points also fluctuate in time (column `step`).

Using a simple linear model with both time (`step`) and variable (`name`) fixed effects, we obtain the following results.

```
Estimation:  OLS
Dep. var.: log_demand, Fixed effects: name+step
Inference:  CRV1
Observations:  1450

| Coefficient   |   Estimate |   Std. Error |   t value |   Pr(>|t|) |   2.5 % |   97.5 % |
|:--------------|-----------:|-------------:|----------:|-----------:|--------:|---------:|
| log_price     |     -2.972 |        0.004 |  -680.195 |      0.000 |  -2.991 |   -2.953 |
---
RMSE: 0.003  Adj. R2: 1.0  Adj. R2 Within: 1.0
```

Required Packages

```
pyfixest==0.10.10.0
seaborn==0.13.0
eerily==0.2.1
```

{...} **Code**

Open in Colab

```
import numpy as np
import pandas as pd
import random

from pyfixest.estimation import feols
import seaborn as sns; sns.set()

import matplotlib.pyplot as plt

from eerily.generators.elasticity import ElasticityStepper, LinearElasticityParams
from eerily.generators.naive import (
    ConstantStepper,
    ConstStepperParams,
    SequenceStepper,
    SequenceStepperParams,
)

from eerily.generators.utils.choices import Choices

# %% [markdown]
# ## Generate Data

# %%
def create_one_article(
    elasticity_value, length, article_id, initial_condition,
    log_prices, first_step=0
):

    es = ElasticityStepper(
        model_params=LinearElasticityParams(
            initial_state=initial_condition,
            log_prices=iter(log_prices),
            elasticity=iter([elasticity_value + (random.random() - 0.5)/10] * length),
            variable_names=["log_demand", "log_price", "elasticity"],
        ),
        length=length
    )

    ss = SequenceStepper(
        model_params=SequenceStepperParams(
            initial_state=[first_step], variable_names=["step"], step_sizes=[1]
        ),
        length=length
    )
    cs = ConstantStepper(
        model_params=ConstStepperParams(initial_state=[article_id], variable_names=["name"]),
        length=length
    )
```

```
    return (es & ss & cs)


initial_condition = {"log_demand": 3, "log_price": 1, "elasticity": None}

length_1 = 200
length_2 = 400
length_3 = 850

log_price_choices_1 = Choices(elements=[1,1.1, 1.2, 1.3, 1.4, 1.5])
log_price_choices_2 = Choices(elements=[1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9])
log_price_choices_3 = Choices(elements=[2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8])

log_prices_1 = [next(log_price_choices_1) for i in range(length_1)]
log_prices_2 = [next(log_price_choices_2) for i in range(length_2)]
log_prices_3 = [next(log_price_choices_3) for i in range(length_3)]


data_gen = (
    create_one_article(elasticity_value=-3, length=length_1, article_id="article_1",
initial_condition=initial_condition, log_prices=log_prices_1)
    + create_one_article(elasticity_value=-3, length=length_2, article_id="article_2",
initial_condition=initial_condition, log_prices=log_prices_2)
    + create_one_article(elasticity_value=-3, length=length_3, article_id="article_3",
initial_condition=initial_condition, log_prices=log_prices_3)
)

# %%
df = pd.DataFrame(list(data_gen))


# %% [markdown]
# ## Visualizations

# %%
fig, ax = plt.subplots(figsize=(10, 6.18))
sns.scatterplot(
    df,
    x="log_price",
    y="log_demand",
    hue="step",
    style="name"
)

# %% [markdown]
# ## Estimation

# %%
fit_feols = feols(
    fml="log_demand ~ log_price | name + step",
    data=df
)

# %%
fit_feols.summary()
```